

Lab 2. Ticket implementer and the harness

The centre of the workshop. 55 minutes. Never cut.

Already ticket in. A green rubric out.

25 minutes of typing. Fill `harness.py`. Nothing else.

Work from `labs/lab2_implementer`.

Spillwave Solutions | spillwave.com

Why this lab exists

The loop you just built will lie to you. Edit forever. Declare victory on red. Stuff the window.

A true story from this repo: seven tests, green on every run, testing the wrong tree. The conftest put the finished answer on `sys.path` ahead of the work copy.

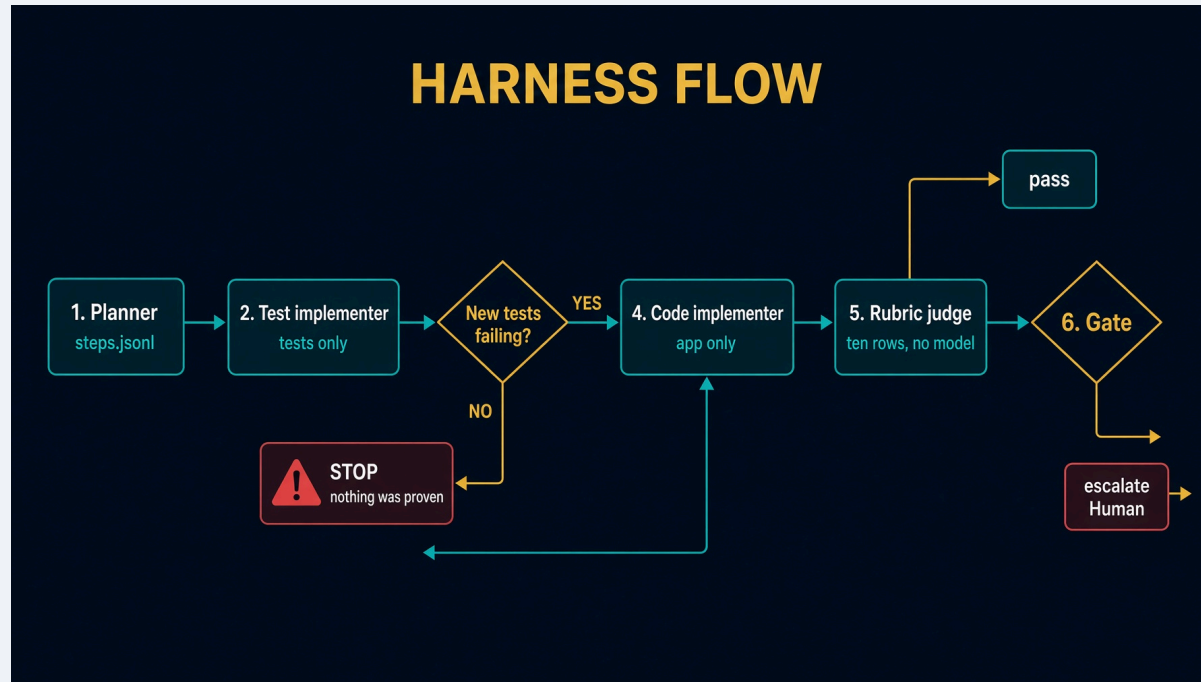
A check that reports success while measuring the wrong thing is worse than no check.

Artifact. A reusable evaluation harness: tests first, prove them red, code until green, judge, gate.

Learning objectives

- Implement `red_gate` so only **new** failing ids count
- Implement `score_attempt` as one call to `rubric.score`
- Implement `run_loop` so Python counts retries, not the model
- Configure two doers with disjoint write scope
- Validate `--doer none` escalates (honesty) and `--doer reference` can pass
- Troubleshoot the stall of computing rubric rows by hand

Starting architecture



You fill three function bodies in `harness.py`. Graph Engineering: each criterion becomes a test step and a code step in `steps.jsonl`.

There is no `loops/` package. Do not recreate it.

Prerequisites

```
cd labs/lab2_implementer
```

Pick one tool and paste its prompt:

```
claude -p "$(cat prompts/claude-code.md)"  
# or: codex exec "$(cat prompts/codex.md)"  
# or: grok -p "$(cat prompts/grok-build.md)"  
# or: opencode run "$(cat prompts/opencode.md)"
```

CRM clone from Lab 1 should already sit at `../..work/northwind-field-crm`.

`.claude/settings.json` **denies** writes to `loops/`, `scripts/`, and `work/**/tests/**`.

The stub. Three functions. The order is the lesson

```
def red_gate(before: RunResult, after: RunResult) -> set[str]:  
    raise NotImplementedError("fill me in")  
  
def score_attempt(contract: Contract, **evidence) -> rubric.Score:  
    raise NotImplementedError("fill me in")  
  
def run_loop(contract: Contract, budget: int = 3) -> dict:  
    raise NotImplementedError("fill me in")
```

Order, from the module docstring:

```
tests first -> prove them red -> code until green -> judge -> gate
```

`red_gate`. **New failing ids, not any failing ids**

A test that already existed and still fails is not proof of a new contract.

A test that passes before any code exists proves nothing.

An empty result must stop the loop.

Body Rick types (and the Deep Agents port ships):

```
def red_gate(before, after) -> set[str]:  
    seen = before.junit.passed_ids | before.junit.failed_ids  
    return {tid for tid in after.junit.failed_ids if tid not in seen}
```

The DA core names this `implementer._new_test_ids`.

score_attempt. **One line. Do not compute rows**

"The tests passed" is one row of ten. Absent kwargs become failing rows on purpose.

```
def score_attempt(contract: Contract, **evidence) -> rubric.Score:
    return rubric.score(contract=contract, **evidence)
```

That is the whole function. The common stall is writing ten `if` blocks.

#	ROW	EVIDENCE
1	tests_ran	junit exists and is non-empty
2	tests_passed	failures+errors == 0
3	red_first	red_ids non-empty
4	coverage_floor	line-rate vs floor (80)
5	criteria_covered	every criterion has a passing test
6	steps_done	no open plan steps
7	ui_has_e2e	UI files changed ⇒ e2e green
8	format_clean	format task ok
9	lint_clean	lint task ok
10	write_scope	no violations (and scope was checked)

`run_loop`. Python holds the retries

```
def run_loop(contract, budget: int = 3, ticket_id: str = "T001", doer: str = "reference") -> dict:
    return implementer.run(
        repo=contract.repo, ticket_id=ticket_id, budget=budget, doer=doer,
    )
```

Saturday this folder has no `implementer.py`. Rick types a body that calls `gates.decide` in a `while` loop, or you watch him fill it. The shipped eight-step loop lives in `solutions/sol2_implementer_deep_agents/implementer.py`.

Pass `doer` through. Hardcoding `reference` makes `--doer none` impossible.

Two doers. Scope is a missing path

From `contract.py` defaults:

```
planner          write_allow: steps.jsonl
test_implementer write_allow: tests/**
code_implementer write_allow: app/**, src/**  write_deny: tests/**
judge            write_allow: []              write_deny: **
```

Deny always beats allow. Empty allow permits nothing. Judge has no `write` method.

The code implementer cannot weaken a test, not because it was told not to, but because it holds no write path to one.

`gates.decide`. **Three exits, no fourth**

1. Rubric green (`judge_done` is `None`) → pass
2. Rubric green and final judge agrees → pass
3. Rubric green, judge says not done → escalate
4. `signature == previous_signature` → escalate (not converging)
5. money budget spent → escalate
6. iteration budget spent → escalate
7. else → retry, `final_attempt` if the next one is last

`signature` is the tuple of **failed row names**, not the wording. Two equal signatures mean the last attempt changed nothing.

Commands. Live first.

Saturday self-check, from `labs/lab2_implementer`:

```
task test    # python3 -c "import harness; print('ok')"
```

Instructor demo, from the Deep Agents driver. Skills are mounted. Python owns the red gate.

```
cd ../../solutions/sol2_implementer_deep_agents
task test-setup
task test
task e2e
task table      # judge writes must print no
task run -- --ticket T001 --doer none
task run -- --ticket T001 --doer reference
```

`--doer none` and `--doer reference` need no key. `--doer deep` needs `task setup` and `ANTHROPIC_API_KEY`.

After class: `.agents/skills/test-ticket-implementer/` plus `HOW_TO_RUN.md` in that folder.

Expected results

`--doer none` writes nothing. The red gate must escalate:

```
gate: escalate
reason: red gate: no new test was observed failing. A test that passes before any code exists proves nothing.
```

If this run were green, the harness would be lying.

`--doer reference` copies known-good into `tests/**` then `app/**`, each phase bound by that role's WriteScope. Expect ten PASS rows and `gate: pass`.

Receipt. Three claims or nothing

`scripts/receipt.py` writes `.harness/receipt.json`.

1. Suite passed (exit 0 **and** readable junit **and** no failures)
2. Ran against **this tree** (`tree_hash` of tracked plus untracked)
3. Ran **after** the newest source edit

A zero exit code with no test report is the silent-skip bug wearing a green shirt.

The push-gate refusal you hit live lives on the CRM clone as a Claude Code hook, not under `labs/lab2_implementer/.claude/`. Read the refusal. It is the lesson.

Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Computing ten rows by hand	stall on <code>score_attempt</code>	<code>return rubric.score(contract=contract, **evidence)</code>
Returning all failed ids	<code>red_gate</code> used after <code>.failed</code>	<code>subtract</code> before <code>ids</code>
Edited <code>loops/</code> or <code>rubric.py</code>	ignored the prompt	fill only <code>harness.py</code>
<code>task loop:implementer</code> missing	engine deleted	<code>task run</code> from <code>sol2_implementer_deep_agents</code>
<code>--doer</code> none is green	red gate not stopping	empty new-ids must escalate
<code>import _root</code> fails	leftover TROUBLESHOOTING	lab stub does not import it

Fall behind

There is no drop-in `harness.py`. Watch Rick finish. Save first:

```
cp harness.py harness.py.my-attempt
```

See `FALL-BEHIND.md`. Restore with `git checkout -- harness.py`.

Take-home, not Saturday: issues 118. `labs/takehome/deep-agents/loop.py` and `labs/takehome/agent-sdk/loop.py` fill build and run. Those ports are a different runtime, not a drop-in for this stub.

Recap

What we built. Three functions that make a loop honest: new red ids, a ten-row rubric, Python-owned retries.

Takeaways

1. Two doers. Disjoint scope. The judge has no write method.
2. A test that passes before any code exists proves nothing.
3. "The tests passed" is one row of ten.
4. Same signature twice is stop.
5. A receipt proves three things, or it proves nothing.